Naive Bayes

Turning evidence into probabilities



The essay detector puzzle

A college uses a detector to flag copied essays. Your friend Aman’s essay just got flagged.

2%

of essays are actually copied

95%

of copied essays get flagged

5%

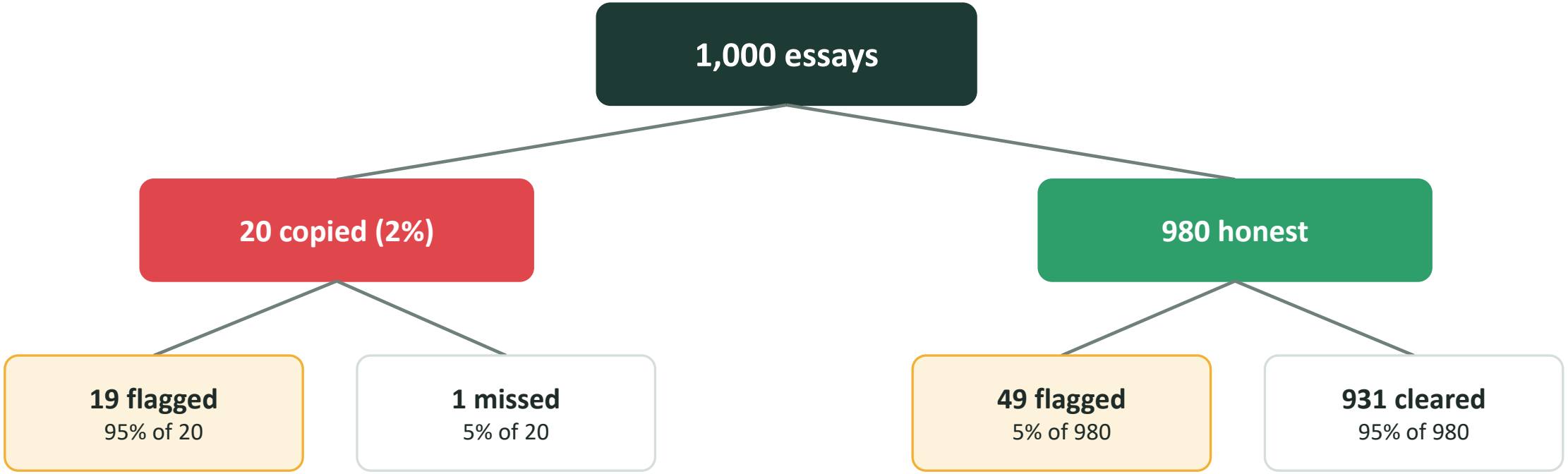
of honest essays get flagged by mistake

Quick poll: how likely is it that Aman really copied?

- **About 95%**
- **About 75%**
- **About 50%**
- **Less than 30%**

Count it out: imagine 1,000 essays

Turning percentages into people makes the answer obvious.



Flagged essays: 19 + 49 = 68. Copied among them: 19 ÷ 68 ≈ 28%

Our route today

Five stops, each answering the question the previous one leaves open.

1

Probability

Counting, and zooming into groups

2

Bayes' rule

Flipping a probability around

3

Naive Bayes

Many features at once, by hand

4

Spam filter

The same idea for words

5

In practice

Numbers, scikit-learn and limits

Riya's prediction

Stop 1 of 5

Probability is just counting

1

Before Bayes, two small ideas: counting what fraction of a group has a property, and zooming into a smaller group first.

Meet 10 students from last year

Three habits we know about each one, and whether they passed.

ID	Coaching	Assignments	Phone at night	Result
A01	Yes	On time	No	Pass
A02	Yes	Late	Yes	Fail
A03	Yes	On time	No	Pass
A04	No	On time	Yes	Fail
A05	Yes	On time	Yes	Pass
A06	No	Late	Yes	Fail
A07	Yes	On time	No	Pass
A08	No	On time	Yes	Pass
A09	No	Late	No	Fail
A10	No	On time	No	Pass

P(Pass)

6 passed out of 10 = 0.6

Every probability in this lesson is a count divided by a count. No formulas to memorise yet.

Conditional probability: zoom into a group

The bar "|" means "given that". First filter the rows, then count.

ID	Coaching	Assignments	Phone at night	Result
A01	Yes	On time	No	Pass
A02	Yes	Late	Yes	Fail
A03	Yes	On time	No	Pass
A04	No	On time	Yes	Fail
A05	Yes	On time	Yes	Pass
A06	No	Late	Yes	Fail
A07	Yes	On time	No	Pass
A08	No	On time	Yes	Pass
A09	No	Late	No	Fail
A10	No	On time	No	Pass

`P(Pass | Coaching = Yes)`

Among the 5 coached students (amber), 4 passed: 4/5 = 0.8

`P(Coaching = Yes | Pass)`

Among the 6 who passed, 4 were coached: 4/6 = 0.67. Not the same number! The order matters.

Your turn: zoom and count

One minute. Use the table.

ID	Coaching	Assignments	Phone at night	Result
A01	Yes	On time	No	Pass
A02	Yes	Late	Yes	Fail
A03	Yes	On time	No	Pass
A04	No	On time	Yes	Fail
A05	Yes	On time	Yes	Pass
A06	No	Late	Yes	Fail
A07	Yes	On time	No	Pass
A08	No	On time	Yes	Pass
A09	No	Late	No	Fail
A10	No	On time	No	Pass

1. `P(Phone = Yes | Fail)`

   `= _____`

2. `P(Fail | Phone = Yes)`

   `= _____`

3. `P(Assignments = On time | Pass)`

   `= _____`

Stop 2 of 5

Flipping a probability around

2

We often know P(evidence | cause) but want P(cause | evidence).
Bayes' rule converts one into the other.

Bayes' rule

Named after Thomas Bayes, an 18th-century English minister and mathematician.

$$P(\text{class} \mid \text{evidence}) = P(\text{evidence} \mid \text{class}) \times P(\text{class}) \div P(\text{evidence})$$

Posterior

`P(class | evidence)`

What we believe after seeing the evidence. This is what we want.

Likelihood

`P(evidence | class)`

How typical this evidence is for that class.

Prior

`P(class)`

How common the class is before seeing anything.

Evidence

`P(evidence)`

How common the evidence is overall. Just rescales.

Worked example: check it on our table

We counted P(Pass | Coaching = Yes) = 4/5 directly. Does Bayes' rule agree?

1. **Likelihood** — P(Coaching = Yes | Pass) = 4/6 = 0.667
2. **Prior** — P(Pass) = 6/10 = 0.6
3. **Evidence** — P(Coaching = Yes) = 5/10 = 0.5
4. **Posterior** — **0.667 × 0.6 ÷ 0.5 = 0.8**

0.8

Same as counting 4 out of 5 directly.

With a small table you could just count. Bayes' rule matters when you can't: lots of features, or only the likelihoods are known.

Stop 3 of 5

Many features at once

3

Riya has three habits we know about. How do we combine all three pieces of evidence?

Riya, and a counting problem

Riya attends coaching, submits on time, and uses her phone late at night.

R

Riya

Coaching: Yes Assignments: On time Phone at night: Yes

ID	Coaching	Assignments	Phone at night	Result
A01	Yes	On time	No	Pass
A02	Yes	Late	Yes	Fail
A03	Yes	On time	No	Pass
A04	No	On time	Yes	Fail
A05	Yes	On time	Yes	Pass
A06	No	Late	Yes	Fail
A07	Yes	On time	No	Pass
A08	No	On time	Yes	Pass
A09	No	Late	No	Fail
A10	No	On time	No	Pass

Only one student (A05) matches all three habits exactly.

One student can’t give a reliable probability. With 20 yes/no habits there are over a million combinations, so most new students would match nobody.

The naive shortcut: treat features as independent

Within each class, pretend the habits don’t influence each other. Then just multiply.

$$P(\text{Coaching}, \text{On time}, \text{Phone} \mid \text{Pass}) \approx P(\text{Coaching} \mid \text{Pass}) \times P(\text{On time} \mid \text{Pass}) \times P(\text{Phone} \mid \text{Pass})$$

Why it helps

Each small probability uses one column at a time, so every student counts. No exact matches needed.

Why “naive”

Habits are rarely independent: students who take coaching may also submit on time. We ignore that. Surprisingly, the predictions are often still good.

Step 1: count a likelihood table

One column at a time, within each class. 6 students passed, 4 failed.

Feature value	Given Pass (6)	Given Fail (4)
Coaching = Yes	4/6	1/4
Coaching = No	2/6	3/4
Assignments = On time	6/6	1/4
Assignments = Late	0/6	3/4
Phone at night = Yes	2/6	3/4
Phone at night = No	4/6	1/4

Priors

P(Pass) = 6/10

P(Fail) = 4/10

This table is the whole trained model. fit() for Naive Bayes is just counting.

Step 2: score each class for Riya

Prior, then one likelihood per feature in order: coaching, assignments, phone.

Score(Pass)

$$6/10 \times 4/6 \times 6/6 \times 2/6$$

= 0.133

Score(Fail)

$$4/10 \times 1/4 \times 1/4 \times 3/4$$

= 0.019

Late-night phone use counts against her (2/6 vs 3/4), but coaching and on-time assignments count strongly for her.

Step 3: normalise. Will Riya pass?

Divide each score by the total so the two add up to 100%.

$$P(\text{Pass} \mid \text{Riya}) = 0.133 \div (0.133 + 0.019) \approx 88\%$$



The whole algorithm: count a table, multiply, compare, normalise. No distances, no trees, no searching.

Check: scikit-learn's CategoricalNB, with smoothing switched off, gives exactly 87.7%.

Your turn: score Kabir

Kabir: Coaching Yes, Assignments On time, Phone at night No.

1. Score(Pass) = 6/10 × ___ × ___ × ___ = _____
2. Score(Fail) = 4/10 × ___ × ___ × ___ = _____
3. P(Pass | Kabir) = _____
4. Is Kabir more or less likely to pass than Riya? Why?

Feature value	Pass	Fail
Coaching = Yes	4/6	1/4
On time	6/6	1/4
Phone = No	4/6	1/4

The rows you need, copied from the likelihood table.

The zero problem: meet Meera

Meera: Coaching Yes, Phone at night No, but she submitted Late. No passing student was ever late.

Score(Pass)

$$6/10 \times 4/6 \times 0/6 \times 4/6$$

= 0 (!)

Score(Fail)

$$4/10 \times 1/4 \times 3/4 \times 1/4$$

= 0.019

Naive Bayes says: 100% Fail. One zero wipes out all the other evidence, however strong. Coaching and no phone don’t count at all.

Laplace smoothing: add 1 to every count

Pretend each feature value was seen once more in every class. No count can be zero any more.

P(value | class) = (count + 1) ÷ (class size + number of possible values)

Meera's feature	Pass, smoothed	Fail, smoothed
Coaching = Yes	(4+1)/(6+2) = 5/8	(1+1)/(4+2) = 2/6
Assignments = Late	**(0+1)/(6+2) = 1/8**	(3+1)/(4+2) = 4/6
Phone = No	(4+1)/(6+2) = 5/8	(1+1)/(4+2) = 2/6

```
Score(Pass) = 0.6 × 5/8 × 1/8 × 5/8 = 0.0293
Score(Fail) = 0.4 × 2/6 × 4/6 × 2/6 = 0.0296
```

Meera: 50% Pass

Not a certain fail any more. A coin toss: her good habits now count.

Stop 4 of 5

The same idea for words

4

Naive Bayes became famous as a spam filter. Words are just features.

Emails as a bag of words

Ignore grammar and word order. Just count which words appear.

Spam

win money now

win free prize

free money offer

Not spam

project meeting today

class notes today

free time today

Step 1: count words per class

Each class has 9 words in total. The vocabulary has 12 different words.

Word	Spam	Not spam
win	2	0
money	2	0
free	2	1
now	1	0
prize	1	0
offer	1	0

Word	Spam	Not spam
today	0	3
project	0	1
meeting	0	1
class	0	1
notes	0	1
time	0	1

With smoothing:

```
P(word | class) = (count + 1) ÷ (9 + 12)
```

Every word in the vocabulary gets at least 1/21 in each class.

Smoothed probabilities per class

P(word | class) = (count + 1) ÷ (9 + 12). Each column adds up to 21/21.

Word	Spam	Not spam
win	3/21	1/21
money	3/21	1/21
free	3/21	2/21
now	2/21	1/21
prize	2/21	1/21
offer	2/21	1/21

Word	Spam	Not spam
today	1/21	4/21
project	1/21	2/21
meeting	1/21	2/21
class	1/21	2/21
notes	1/21	2/21
time	1/21	2/21

How to read it

win: 3/21 in spam vs 1/21 in not spam, so it points to spam.

today: 1/21 vs 4/21, so it points to not spam.

No zeros: a word never seen in a class still gets 1/21.

Step 2: is “win free money” spam?

Multiply one smoothed probability per word.

win free money

Spam

$0.5 \times 3/21 \times 3/21 \times 3/21$

$\propto 27$

Not spam

$0.5 \times 1/21 \times 2/21 \times 1/21$

$\propto 2$

$$P(\text{spam}) = 27 \div (27 + 2) \approx 93\%$$

A practical detail: add logs, don’t multiply

Real emails have hundreds of words. Multiplying hundreds of tiny numbers breaks the computer.

The problem

Multiply 300 probabilities of about 0.01 each and you get 10^{-600}. Computers round anything below roughly 10^{-308} to exactly 0, so every class scores 0.

The fix

$$\log(a \times b \times c) = \log a + \log b + \log c$$

Adding 300 logs is easy and safe. The class with the bigger log-score still wins. scikit-learn does this for you.

Stop 5 of 5

Numbers, scikit-learn and limits

5

What if features are numbers, like study hours? Then the code, and when not to trust Naive Bayes.

Four flavours in scikit-learn

Choose by the kind of features you have. All live in sklearn.naive_bayes.

Class	Features look like	Likelihood comes from	Today's example
`GaussianNB`	Continuous numbers	A bell curve per class	Hours and mock score
`MultinomialNB`	Counts (word counts)	Smoothed frequencies	The spam filter
`BernoulliNB`	Yes/no (word present or not)	Smoothed presence rates	Spam, presence only
`CategoricalNB`	Categories	Smoothed counts per category	Coaching, assignments, phone

Same API as every model so far: fit, predict, predict_proba. The smoothing setting is called alpha (default 1.0).

The spam filter in a few lines

CountVectorizer builds the bag of words; MultinomialNB does the counting.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

emails = ["win money now", "win free prize", "free money offer",
          "project meeting today", "class notes today",
          "free time today"]
is_spam = [1, 1, 1, 0, 0, 0]

vec = CountVectorizer()
X = vec.fit_transform(emails)          # bag of words
model = MultinomialNB(alpha=1.0).fit(X, is_spam)

new = vec.transform(["win free money", "free money today"])
print(model.predict_proba(new).round(3))
```

Output [not spam, spam]

```
[[0.069 0.931]
 [0.471 0.529]]
```

93% and 53%: exactly the numbers we worked out by hand.

Lab: Naive Bayes on the 240 students

Same split as the decision tree and forest labs, so the scores are comparable.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

df = pd.read_csv("Decision_Tree_Student_Success_Dataset.csv")
X = df[["hours", "mock_score"]]
y = df["passed"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

nb = GaussianNB().fit(X_train, y_train)
print("Test accuracy:", nb.score(X_test, y_test))
print(nb.theta_)      # mean per class and feature
print(nb.var_)        # spread per class and feature
```

Challenges

Compare with the tuned tree (68.3%) and your forest.

Read theta_: do the class means make sense?

Time fit() for all three models. Which is fastest?

Where Naive Bayes fits in your toolkit

Fast, simple, and happy with very little data.

	KNN	Decision tree	Random forest	Naive Bayes
fit() does	Stores data	Searches for questions	Grows many trees	**Counts / averages**
Training speed	Instant	Fast	Slower	**Very fast**
Small datasets	Weak	Overfits easily	OK	**Good**
Text with thousands of words	Poor	Poor	OK	**Excellent**
Reliable probabilities	Rough	Rough	Fair	Often overconfident
Needs scaling	Yes	No	No	No